Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра программного обеспечения информационных технологий

Разработка программного обеспечения для мобильных платформ

ОТЧЕТ

по лабораторной работе №2

Каталог видеоигр и игровых студий с рейтингами

Выполнил
Студент гр. 351002 Михальков А. Д.

Проверил Петровская В. В.

Минск 2026

СОДЕРЖАНИЕ

1 Введение 3
1.1 Функциональные изменения 3
1.1.1 Архитектурная реструктуризация (MVVM) 3
1.1.2 Интеграция с Web API 4
1.1.3 Мониторинг сети и Offline-режим 4
1.1.4 Расширенная фильтрация и UX 5
1.2 Теоретическая основа 6
1.3 Технологии 6
2 Скриншоты работы приложения 8
2.1 Вкладка «Global» и работа с API 8
2.2 Фильтрация по жанрам 9
2.3 Мониторинг сети и кэширование данных 10
3 Заключение 11

1 Введение

Цель работы: Модернизация мобильного приложения путем перехода на современную многослойную архитектуру MVVM, интеграции с глобальными внешними сервисами через Web API, реализации системы мониторинга сетевого статуса и разработки механизмов кэширования данных в локальную базу данных для обеспечения полноценной работы приложения в оффлайн-режиме.

1.1 Функциональные изменения

1.1.1 Архитектурная реструктуризация (MVVM)

В рамках работы произведено полное разделение бизнес-логики и уровня представления. Приложение переведено на архитектуру Model-View-ViewModel (MVVM), что позволило устранить жесткую зависимость между интерфейсом и механизмами получения данных, повысив чистоту и модульность программного кода.

Структура распределения ролей в архитектуре приложения выглядит следующим образом:

1. Model: Инкапсулирует низкоуровневые операции с данными. Уровень данных представлен модулем models/db.js, отвечающим за SQL-запросы к базе данных SQLite, и модулем models/api.js, реализующим асинхронные HTTP-запросы к внешнему серверу через библиотеку Axios. Модели являются «чистыми» компонентами: они не содержат логики состояния интерфейса и возвращают только необработанные данные.
2. ViewModel: Реализована в виде кастомных React-хуков (например, useHomeViewModel, useAuthViewModel). ViewModel выступает в роли посредника, который запрашивает данные у моделей, управляет их состоянием (useState), выполняет фильтрацию (включая поиск и выбор жанров) и отслеживает жизненный цикл экрана (useEffect). Все изменения данных происходят здесь, после чего результат «транслируется» в View.
3. View: Файлы в директории views/ представляют собой визуальные компоненты. Экраны лишились прямого доступа к базе данных и сетевым методам. Теперь они являются «пассивными»: получают готовые переменные и функции-обработчики от своей ViewModel через деструктуризацию. Это позволяет изменять внешний вид приложения без риска нарушения логики обработки данных.

Теперь каждый слой может развиваться независимо: например, замена SQLite на другую СУБД потребует внесения изменений только в слой Model, не затрагивая визуальную часть приложения.

1.1.2 Интеграция с Web API

Реализована вкладка «Global», данные для которой подгружаются из внешнего сервиса RAWG API – крупнейшей открытой базы данных видеоигр.

Техническая реализация интеграции включает следующие аспекты:

1. Сетевое взаимодействие: Запросы к внешнему серверу реализованы в модуле models/api.js с помощью библиотеки Axios. Использование асинхронных функций (async/await) позволяет загружать данные в фоновом режиме, не блокируя основной поток интерфейса.
2. Параметризация запросов: Приложение отправляет GET-запросы к эндпоинту /games, используя персональный API-ключ. Для предоставления пользователю наиболее качественного контента во ViewModel настроены параметры фильтрации: объем выборки (15 объектов) и сортировка по мировому рейтингу (ordering: "-rating").
3. Обработка данных: Полученные от API данные в формате JSON (названия, изображения, даты релизов) автоматически преобразуются в объекты JavaScript и передаются в интерфейс. Для отображения обложек игр используется механизм загрузки изображений по удаленным URL-ссылкам.

Теперь пользователь может сравнивать свои персональные оценки с общемировыми показателями, а также использовать глобальный список для поиска новых игр.

1.1.3 Мониторинг сети и Offline-режим

В приложение добавлена система интеллектуального отслеживания статуса интернет-соединения, что является критически важным для мобильных устройств, работающих в условиях нестабильного приема. При отсутствии сети приложение автоматически переходит в оффлайн-режим, мгновенно адаптируя логику работы с данными и уведомляя пользователя о текущем статусе через специальный информационный баннер.

Основные компоненты реализации включают:

1. Активный мониторинг: С помощью библиотеки NetInfo (@react-native-community/netinfo) во ViewModel реализован «слушатель» (listener) сетевых событий. Приложение в реальном времени реагирует на включение/выключение Wi-Fi или мобильного интернета, обновляя глобальное состояние isOffline.

2. Визуальная индикация: Для улучшения UX в верхней части главного экрана реализован баннер «OFFLINE MODE». Благодаря системе локализации (i18next), текст баннера автоматически переводится на выбранный пользователем язык, обеспечивая понятную обратную связь в любых условиях.
3. Гибридная логика данных (Offline-First): Для обеспечения бесперебойного доступа к информации реализован механизм кэширования внешних данных в локальную базу данных SQLite. Логика работы разделена на два сценария:
    - В режиме онлайн: При каждом успешном запросе к RAWG API приложение вызывает метод saveApiCache, который полностью очищает старый кэш в таблице api_cache и записывает туда свежие данные. Это гарантирует актуальность информации.
    - В режиме оффлайн: Если сетевой запрос невозможен, ViewModel блокирует обращение к внешнему серверу и инициирует чтение из таблицы api_cache. Таким образом, вкладка «Global» не остается пустой, а отображает последние сохраненные результаты.

Реализация данных механизмов позволила добиться высокой отказоустойчивости приложения. Пользователь сохраняет возможность полноценного просмотра мировых рейтингов и взаимодействия с каталогом даже в условиях полного отсутствия связи.

1.1.4 Расширенная фильтрация и UX

Для обеспечения более гибкого взаимодействия с каталогом и повышения удобства использования в приложение был введен функционал расширенной фильтрации и современные механизмы обновления контента.

Ключевые улучшения включают:

1. Динамическая фильтрация по жанрам: На вкладке мировых игр реализована горизонтальная панель фильтров (Action, RPG, Indie, Adventure и др.). В отличие от простой локальной фильтрации, данный механизм работает через динамические запросы к API. При выборе пользователем конкретного жанра ViewModel перехватывает событие, обновляет параметры запроса и инициирует повторную загрузку данных с сервера RAWG, передавая уникальный идентификатор жанра. Это позволяет пользователю точечно исследовать интересующие его сегменты игрового рынка, получая актуальные результаты напрямую из глобальной базы.
2. Механизм Pull-to-Refresh: Реализован стандартный для мобильных платформ жест «обновление потяжкой вниз». Технически это выполнено через интеграцию свойств refreshing и onRefresh в компонент FlatList. При активации жеста вызывается глобальный метод обновления данных

во ViewModel, который синхронно актуализирует локальную базу данных, обновляет состояние студий и выполняет свежий сетевой запрос к API с последующей перезаписью оффлайн-кэша. Это дает пользователю возможность принудительно обновить информацию в любой момент времени.

1.2 Теоретическая основа

В данной работе акцент смещен на асинхронное взаимодействие и чистую архитектуру.

Использование паттерна MVVM позволяет добиться высокой степени независимости компонентов. ViewModel выступает в роли посредника: она запрашивает данные у моделей, применяет к ним бизнес-правила (например, ограничение рейтинга до 5.0 или фильтрация по строке поиска) и предоставляет View уже готовый для отображения объект.

Для работы с сетью выбрана библиотека axios, обеспечивающая удобный интерфейс для выполнения HTTP-запросов. Система кэширования строится на принципе "Cache-aside": при успешном сетевом ответе данные записываются в таблицу api_cache, а при ошибке сети (или статусе offline) – считываются из неё.

1.3 Технологии

В качестве технологий для выполнения данной работы были выбраны следующие:

1. Axios: HTTP-клиент на основе Promise для браузера и node.js. В проекте он выбран в качестве альтернативы стандартному fetch благодаря удобному механизму передачи параметров и автоматической трансформации данных из формата JSON в объекты JavaScript.
2. NetInfo (@react-native-community/netinfo): Модуль для мониторинга состояния сетевого подключения. Он предоставляет API для получения текущего типа соединения и позволяет подписываться на события изменения статуса сети в реальном времени.
3. RAWG API: Крупнейшая мировая база данных видеоигр, предоставляющая RESTful API. Сервис используется как основной источник данных для глобального каталога, предоставляя информацию о более чем 500 000 игр.
4. React Custom Hooks: Использование пользовательских хуков стало технической базой для реализации паттерна MVVM. Логика работы с API, БД и состоянием сети инкапсулирована в хуки useHomeViewModel, useAuthViewModel и др., что позволило отделить бизнес-логику от жизненного цикла UI-компонентов.

5. SQLite: Помимо хранения пользовательских данных, возможности встроенной БД были расширены для реализации Persistence Cache. Создание специализированных таблиц для кэширования ответов API позволило реализовать функционал оффлайн-доступа.
6. ActivityIndicator & Pull-to-Refresh API: Встроенные нативные инструменты React Native, использованные для визуализации асинхронных процессов загрузки и обновления данных, обеспечивающие привычный пользователю мобильный UX.

2 Скриншоты работы приложения

2.1 Вкладка «Global» и работа с API

На главном экране добавлена третья вкладка. При переключении пользователя на данный раздел уровень представления взаимодействует с соответствующей ViewModel, которая инициирует асинхронный сетевой запрос к внешнему серверу RAWG API через модуль api.js.

В процессе ожидания ответа от сервера, который выполняется в фоновом режиме с использованием библиотеки Axios, на экране отображается системный индикатор загрузки ActivityIndicator. Это техническое решение информирует пользователя об активном процессе получения данных и обеспечивает плавность интерфейса.

Полученные из сети данные рендерятся в виде вертикального списка, где каждая карточка содержит ключевую информацию о мировых играх: полное название проекта, дату его официального релиза и актуальный мировой рейтинг.

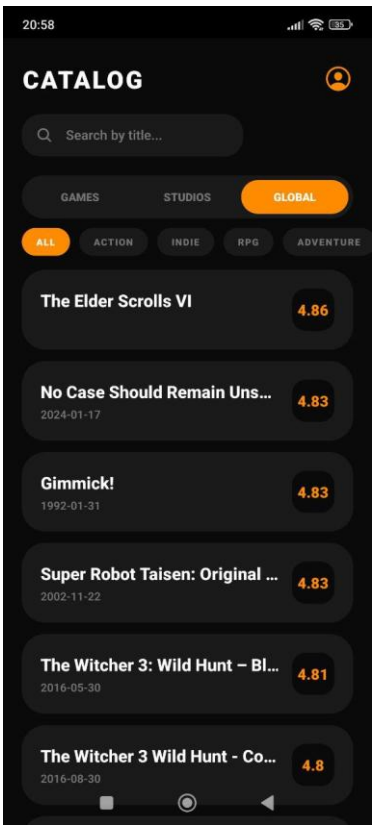

Рисунок 1. Вкладка «Global»

2.2 Фильтрация по жанрам

На вкладке мировых игр добавлена горизонтальная лента фильтров. Данный элемент интерфейса реализован с использованием компонента ScrollView с горизонтальной ориентацией, что позволяет пользователю перемещаться между доступными жанрами.

При выборе конкретной категории, например «INDIE», уровень представления взаимодействует с ViewModel, которая мгновенно обновляет идентификатор выбранного жанра в своем состоянии. Это изменение автоматически инициирует новый цикл загрузки данных: слой Модели формирует уточненный сетевой запрос к API, передавая выбранный жанр в качестве параметра фильтрации. После получения асинхронного ответа список игр мгновенно актуализируется, отображая только те проекты, которые относятся к выбранному направлению, что наглядно подтверждается списком игр на экране.

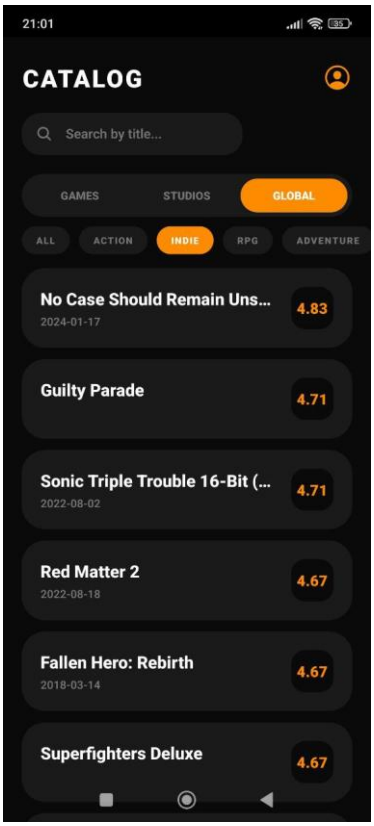

Рисунок 2. Вкладка «Global» с выбранным фильтром

2.3 Мониторинг сети и кэширование данных

Техническая реализация данного функционала базируется на использовании библиотеки NetInfo, которая интегрирована во ViewModel главного экрана и выступает в роли активного слушателя сетевых событий.

При отключении Wi-Fi или мобильной передачи данных приложение мгновенно реагирует на изменение статуса сети, отображая в верхней части интерфейса баннер с надписью «OFFLINE MODE». Появление этого элемента управляется внутренним состоянием ViewModel, которое синхронизируется с уровнем представления и автоматически переводится на выбранный пользователем язык благодаря модулю локализации.

Параллельно с визуальным оповещением срабатывает логика переключения источников данных: понимая, что запрос к внешнему RAWG API невозможен, ViewModel блокирует сетевой уровень и обращается к локальной базе данных SQLite.

Вкладка Global не остается пустой, а продолжает отображать полноценный список игр. Это достигается за счет использования таблицы api_cache, в которую приложение записывает результаты последнего успешного сетевого ответа во время нахождения в онлайн-режиме.

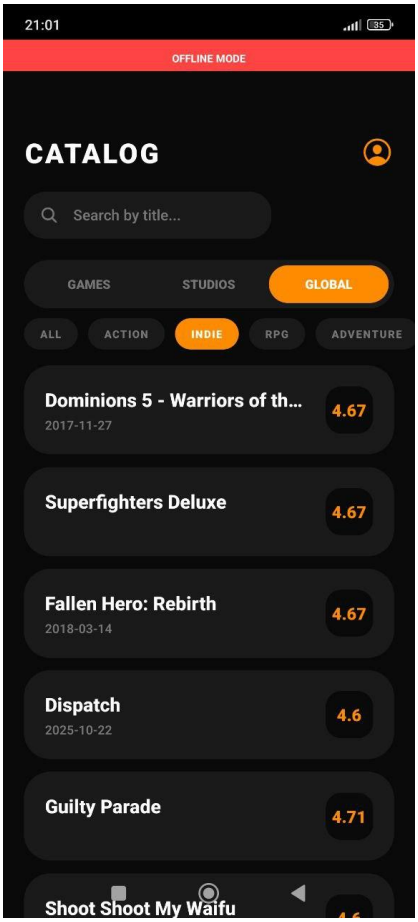

Рисунок 3. Вкладка «Global» с выключенным интернет-соединением

3 Заключение

В ходе выполнения лабораторной работы была успешно осуществлена комплексная модернизация мобильного приложения, ключевым этапом которой стало внедрение архитектурного паттерна MVVM. Структурная реорганизация проекта и разделение его на независимые слои (Model, View, ViewModel) позволили полностью изолировать сложную бизнес-логику и механизмы работы с данными от уровня представления. Это существенно повысило надежность, читаемость и масштабируемость программного кода, подготовив приложение к дальнейшему расширению функционала без риска нарушения целостности интерфейса.

В рамках работы были глубоко освоены принципы взаимодействия с внешними Web API с использованием библиотеки Axios. Практическая реализация асинхронных запросов, параметризации данных и обработки сетевых исключений позволила превратить изолированное локальное хранилище в динамичный информационный агрегатор, предоставляющий пользователю доступ к актуальной мировой статистике в режиме реального времени.

Особое внимание было уделено отказоустойчивости продукта. Реализованный механизм кэширования внешних данных в локальную базу данных SQLite в сочетании с системой активного мониторинга сети на базе модуля NetInfo обеспечили приложению возможность полноценного функционирования в условиях нестабильного или полностью отсутствующего интернет-соединения. Данный подход (Offline-First) является критически важным для современных мобильных решений, гарантируя сохранение доступности контента в любых обстоятельствах.

Итогом работы стало создание технологичного и стабильного мобильного клиента, спроектированного в соответствии с актуальными стандартами индустрии мобильной разработки.